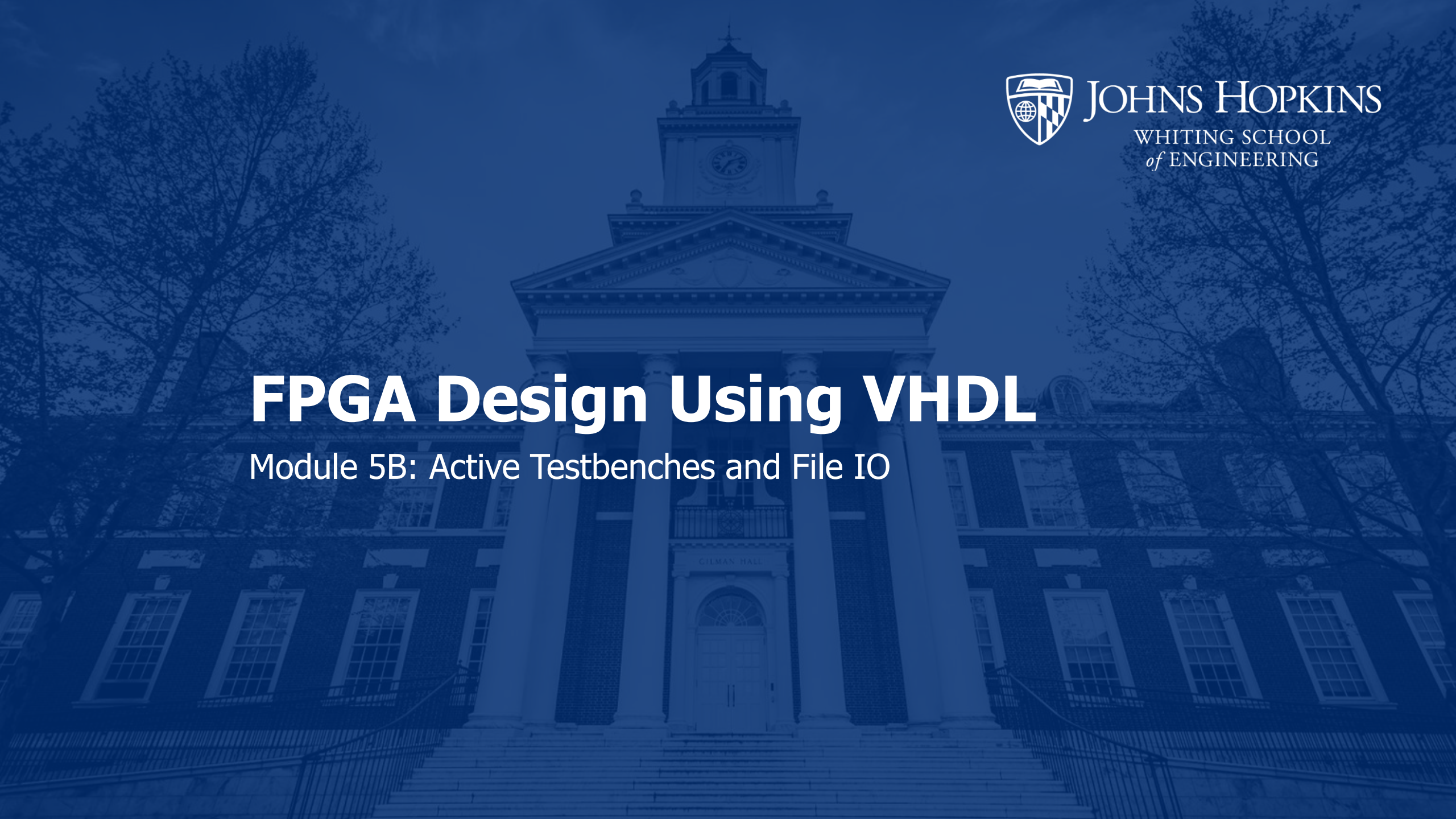


FPGA Design Using VHDL

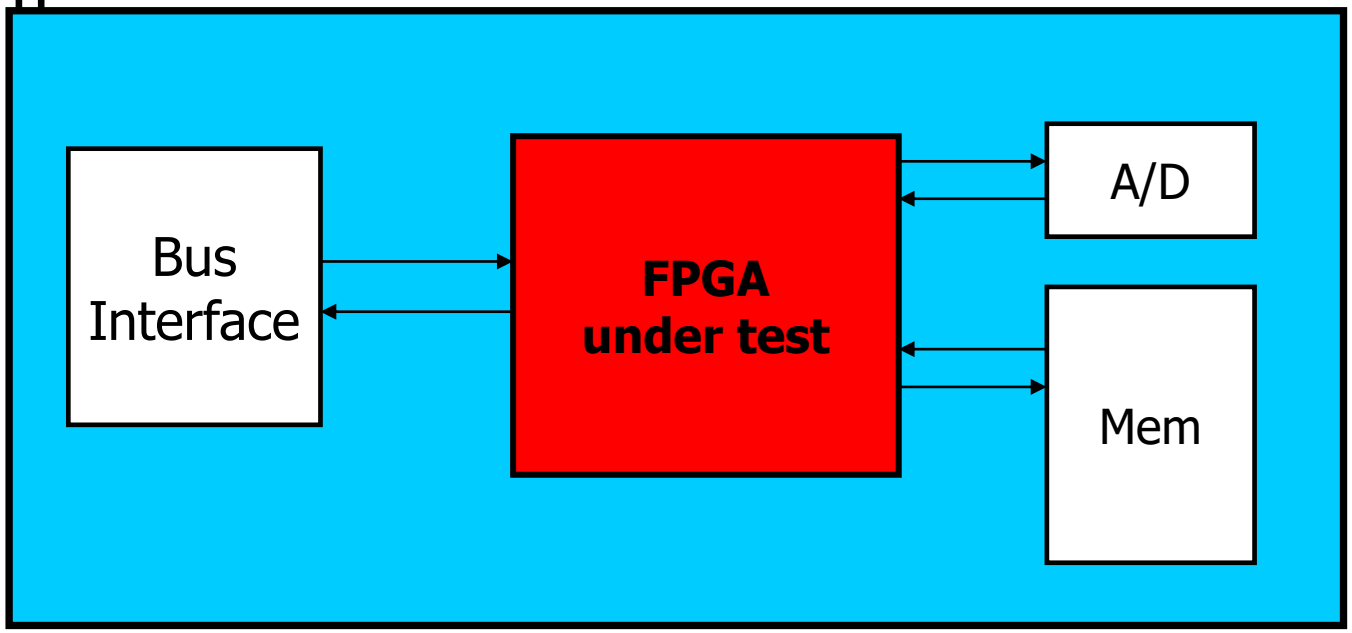
Module 5B: Active Testbenches and File IO

Active Testbench

- Design is frequently in a system which is quite difficult to test by manual forcing of signals
  - If FPGA has two-way communication with other devices, a testbench that actually responds to signals from your device is of great benefit

Example of System for Testbench

- Instead of testing by stimulating all FPGA inputs with test vectors, use VHDL to create models of the A/D, memory, and bus interface, so that the FPGA can be observed in a replica of the actual system



File I/O: Modeling Memory

- Automated testbench for populating memory device and reading back contents
- FILE I/O is a perfect tool for this
  - VHDL testbench can read from a file and can write to a file

File I/O: Automated Testing

- Testbench can use FILE I/O to read a “vector file” which contains a list of all the stimulus in tabular form
- VHDL testbench can contain statements that automatically verify design
  - Doesn’t require designer to verify response using waveforms

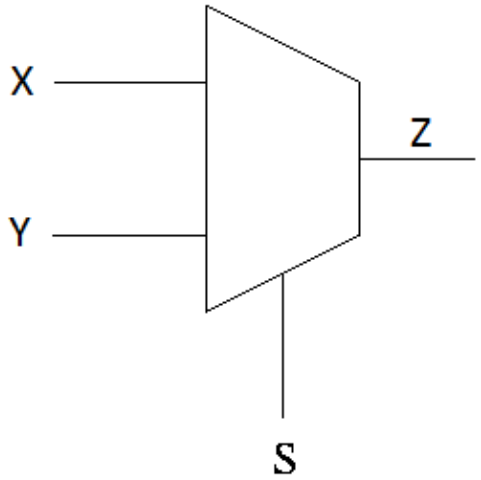
Automated Testing

- Combination of file I/O and asserts are used to:
  - Read a file of test inputs, and desired outputs
  - Stimulate the circuit with the inputs
  - Compare the outputs with the desired outputs from the file using the assert
  - Print out errors and/or stop on assertion failure

Multiplexer Testing File

Generated using another tool:

```
-- order: x is in1, y, is in2, s is sele
-- xys result
   000   0
   001   0
   011   1
   010   0
   110   1
   111   1
   101   0
   100   1
```



Automated Testing Example

```
use std.textio.all;
-- entity omitted
architecture test_bench of mux_test is

  signal in0, in1, sel, z : bit;

begin
    -- entity/architecture for mux.vhd not shown
  DUT : entity mux port map (in0,in1,sel,z);

    -- test process goes here (from next slide)

end test_bench;
```

Automated Testing Example Continued

```
process
   file data_fp : text open read_mode is "mux.dat";
   variable sample : line;
   variable in0_var, in1_var, sel_var, z_var : bit
begin
   while not endfile(data_fp) loop
      readline(data_fp,sample);
      read(sample,in0_var);
      read(sample,in1_var);
      read(sample,sel_var);
      read(sample,z_var); -- expected output from mux.dat
      in0 <= in0_var;
      in1 <= in1_var;
      sel <= sel_var;
      wait for 10 ns;
      assert z = z_var report "z incorrect" severity error;
end loop;
   wait;
end;
```

Assertions

- Used to make sure a condition is true, if not, a report is generated by the simulator.

```
   assert z = z_var report "z   incorrect"
severity error;
```

- Severity can be
  - note, warning, error, failure, fatal
- Simulator can be told to ignore or break on all but fatal (which always breaks)